

EE3621: Digital Signal Processing

Lecture 20: IIR Filter — Lattice Form Realization (III B.Tech EEE)

Comprehensive Lecture Notes & Faculty Textbook Reference

Lecture Plan & Objectives (40 Minutes Breakdown)

- **00:00 – 06:00 (6 mins):** Motivation for Lattice Realizations: The fundamental vulnerability of Direct Form structures to finite word-length coefficient quantization. Inversion of signal flow from FIR forward prediction lattices to All-Pole IIR feedback lattices.
 - **06:00 – 16:00 (10 mins):** Mathematical Foundations of All-Pole IIR Lattice: Signal flow inversion ($f_{m-1}[n] = f_m[n] - k_m g_{m-1}[n-1]$), step-down (Levinson-Durbin) algorithm, reverse polynomials $B_m(z) = z^{-m} A_m(z^{-1})$, and the Schur-Cohn stability criterion ($|k_m| < 1 \forall m$).
 - **16:00 – 25:00 (9 mins):** General IIR Synthesis via Lattice-Ladder Architecture (Gray-Markel Structure): Orthogonality of the backward prediction error basis $g_m[n]$, tapping orthogonal states to synthesize arbitrary zeros via ladder coefficients c_m , and recursive polynomial peeling.
 - **25:00 – 31:00 (6 mins):** Architectural Hardware Complexity Audit: Multipliers, adders, registers, sensitivity analysis, pipelineability, and zero-input limit-cycle suppression.
 - **31:00 – 38:00 (7 mins):** Step-by-Step Worked Numerical Examples: 3rd-order all-pole step-down analysis, complete 2nd-order pole-zero lattice-ladder filter synthesis with verification, and reverse analysis from SFG.
 - **38:00 – 40:00 (2 mins):** Faculty Insights, Common Misconceptions, and University Semester Exam Traps.
-

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Formulate** the all-pole IIR lattice recursion by inverting the signal flow of an FIR forward prediction lattice.
2. **Execute** the recursive step-down (Levinson-Durbin) algorithm to convert direct-form denominator polynomials $A_N(z)$ into reflection coefficients k_m .
3. **Evaluate** the Schur-Cohn stability criterion directly in lattice parameter space ($|k_m| < 1 \forall m \in \{1, \dots, N\}$) without numerical root-finding.

4. **Synthesize** the canonical Gray-Markel Lattice-Ladder architecture for general IIR transfer functions $H(z) = B(z)/A(z)$ possessing arbitrary poles and zeros.
 5. **Derive** the feedforward ladder tap weights c_m by projecting the numerator polynomial onto the orthogonal reverse polynomial basis $B_m(z)$.
 6. **Audit** exact hardware resource requirements (multipliers, adders, delay elements) and contrast them with Direct Form II, Cascade Form, and Parallel Form.
 7. **Explain** the extreme robustness of lattice structures to finite word-length effects (coefficient quantization and suppression of overflow limit cycles).
 8. **Perform** reverse synthesis from a given lattice-ladder signal flow graph to recover the rational transfer function $H(z)$.
-

2. MATHEMATICAL FOUNDATIONS OF IIR LATTICE REALIZATION

2.1 From FIR Forward Prediction to All-Pole IIR Feedback Lattice

In the synthesis of digital filters, the lattice structure holds a unique and prestigious position in speech processing (linear predictive coding), seismic signal processing, and high-precision VLSI implementations. To derive the IIR lattice, we begin by reviewing the fundamental FIR lattice filter.

Recall that an M -stage FIR lattice filter is governed by the forward and backward prediction error recursions:

$$f_m[n] = f_{m-1}[n] + k_m g_{m-1}[n-1] \quad (1)$$

$$g_m[n] = k_m f_{m-1}[n] + g_{m-1}[n-1] \quad (2)$$

with boundary conditions $f_0[n] = g_0[n] = x[n]$, and overall FIR output $y[n] = f_M[n]$. In the Z -domain, defining $A_m(z) = F_m(z)/X(z)$ and $B_m(z) = G_m(z)/X(z)$, the system transfer functions satisfy:

$$A_m(z) = A_{m-1}(z) + k_m z^{-1} B_{m-1}(z) \quad (3)$$

$$B_m(z) = k_m A_{m-1}(z) + z^{-1} B_{m-1}(z) = z^{-m} A_m(z^{-1}) \quad (4)$$

where $B_m(z)$ is the **reverse polynomial** of $A_m(z)$, formed by reversing the order of its coefficients.

Now consider an **all-pole IIR filter** whose transfer function is the reciprocal of an N -th order polynomial:

$$H_{ap}(z) = \frac{Y(z)}{X(z)} = \frac{1}{A_N(z)} = \frac{1}{1 + \sum_{i=1}^N \alpha_N(i) z^{-i}} \quad (5)$$

Cross-multiplying yields the input-output relationship:

$$X(z) = A_N(z)Y(z) \iff x[n] = y[n] + \sum_{i=1}^N \alpha_N(i) y[n-i] \quad (6)$$

This reveals a profound mathematical duality: **the input $x[n]$ to an all-pole IIR filter is identical to the output of an FIR lattice filter whose input is $y[n]$!**

To realize the all-pole IIR filter, we must therefore **invert the signal flow** of the FIR lattice:

- The external input $x[n]$ is injected into stage N : $f_N[n] = x[n]$.
- The all-pole output is tapped at stage 0: $y[n] = f_0[n] = g_0[n]$.
- Rearranging the forward error equation (1) to solve for $f_{m-1}[n]$ in terms of $f_m[n]$:

$$f_{m-1}[n] = f_m[n] - k_m g_{m-1}[n-1] \quad (7)$$

- The backward error equation remains unaltered:

$$g_m[n] = k_m f_{m-1}[n] + g_{m-1}[n-1] \quad (8)$$

Because $f_{m-1}[n]$ is computed by subtracting $k_m g_{m-1}[n-1]$ from $f_m[n]$, every lattice stage contains a **negative feedback loop**. The signal traverses the stages in descending order from $m = N, N-1, \dots, 1$, and terminates at stage 0 where $f_0[n] = g_0[n] = y[n]$.

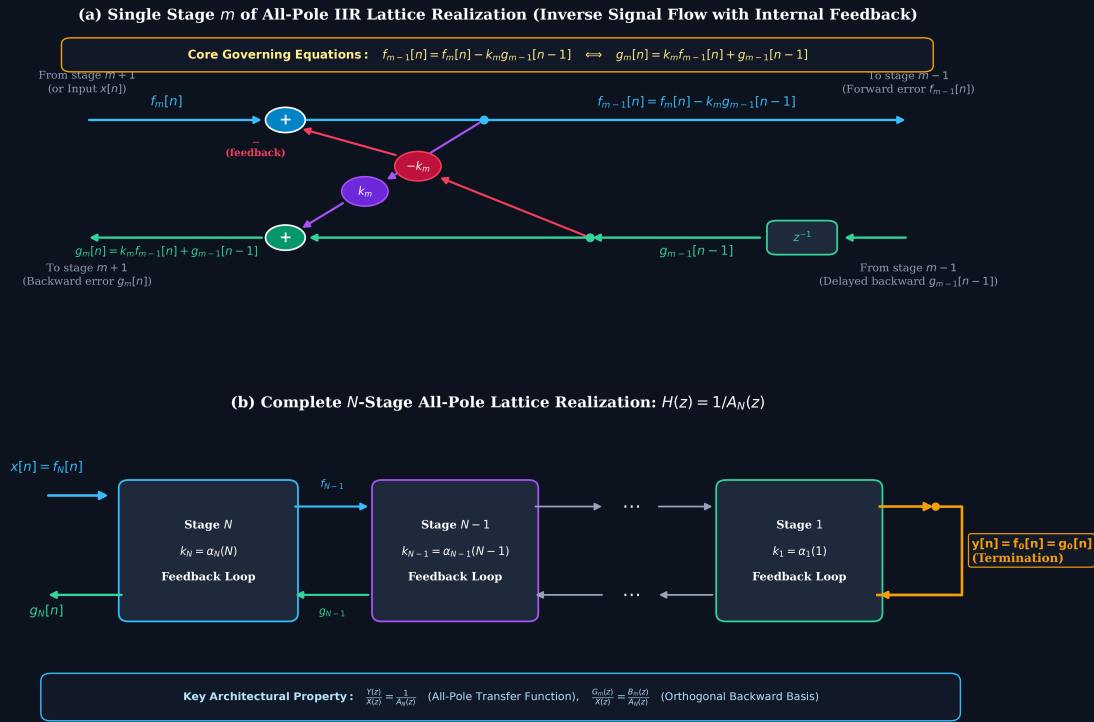


Figure 1: All-Pole IIR Lattice Realization: (a) Single Stage m illustrating the inverted signal flow with internal feedback loop ($f_{m-1}[n] = f_m[n] - k_m g_{m-1}[n-1]$ and $g_m[n] = k_m f_{m-1}[n] + g_{m-1}[n-1]$). (b) Complete N -stage cascade where input $x[n] = f_N[n]$ flows left-to-right to produce output $y[n] = f_0[n] = g_0[n]$, while generating the orthogonal backward error sequence $g_m[n]$.

2.2 The Step-Down Algorithm (Direct Form to Lattice Conversion)

Given the denominator polynomial of an all-pole or general IIR filter:

$$A_N(z) = 1 + \sum_{i=1}^N \alpha_N(i) z^{-i} = 1 + \alpha_N(1) z^{-1} + \dots + \alpha_N(N) z^{-N} \quad (9)$$

we must determine the N reflection coefficients $k_1, k_2, \dots, k_N$ that synthesize this denominator. This is accomplished via the recursive **Step-Down Algorithm** (also known as the Levinson-Durbin recursion in reverse).

From the polynomial recursion (3), evaluating at order m :

$$A_m(z) = A_{m-1}(z) + k_m z^{-1} B_{m-1}(z) \quad (10)$$

Since $B_{m-1}(z) = z^{-(m-1)} A_{m-1}(z^{-1}) = \alpha_{m-1}(m-1) + \dots + z^{-(m-1)}$, multiplying by z^{-1} makes its highest power z^{-m} with coefficient 1. The coefficient of z^{-m} in $A_m(z)$ is therefore:

$$\mathbf{k}_m = \alpha_m(\mathbf{m}) \quad (11)$$

To extract the lower-order polynomial $A_{m-1}(z)$, we write the system of equations for $A_m(z)$ and its reverse $B_m(z)$:

$$A_m(z) = A_{m-1}(z) + k_m z^{-1} B_{m-1}(z) \quad (12)$$

$$B_m(z) = k_m A_{m-1}(z) + z^{-1} B_{m-1}(z) \quad (13)$$

Multiplying (13) by k_m and subtracting from (12):

$$A_m(z) - k_m B_m(z) = (1 - k_m^2) A_{m-1}(z) \quad (14)$$

Dividing by $(1 - k_m^2)$, we obtain the fundamental **Step-Down Equation**:

$$\mathbf{A}_{m-1}(\mathbf{z}) = \frac{\mathbf{A}_m(\mathbf{z}) - \mathbf{k}_m \mathbf{B}_m(\mathbf{z})}{1 - \mathbf{k}_m^2} \quad (15)$$

Equating coefficients of z^{-i} on both sides yields the element-wise formula:

$$\alpha_{m-1}(\mathbf{i}) = \frac{\alpha_m(\mathbf{i}) - \mathbf{k}_m \alpha_m(\mathbf{m} - \mathbf{i})}{1 - \mathbf{k}_m^2}, \quad \mathbf{i} = 1, 2, \dots, m-1 \quad (16)$$

The algorithm executes iteratively for $m = N, N-1, \dots, 2, 1$:

1. Read the reflection coefficient: $k_m = \alpha_m(m)$.
2. Compute the divisor $1 - k_m^2$.
3. Compute $\alpha_{m-1}(i)$ for each $i \in \{1, \dots, m-1\}$ using (16).
4. Decrement m and repeat until $m = 1$.

2.3 2.3 Schur-Cohn Stability Criterion & Decoupled Parameter Space

A perennial challenge in direct form IIR design is stability verification. For an N -th order direct form filter, determining whether all poles lie strictly inside the unit circle requires calculating the roots of $A_N(z) = 0$, an $O(N^3)$ numerical procedure that is highly sensitive to roundoff errors.

In lattice realization, stability verification is governed by the celebrated **Schur-Cohn Theorem**:

Schur-Cohn Stability Theorem: An all-pole IIR digital filter $H(z) = 1/A_N(z)$ is Bounded-Input Bounded-Output (BIBO) stable (all poles satisfy $|p_i| < 1$) **if and only if** all reflection coefficients satisfy:

$$|\mathbf{k}_m| < 1 \quad \forall \mathbf{m} \in \{1, 2, \dots, \mathbf{N}\} \quad (17)$$

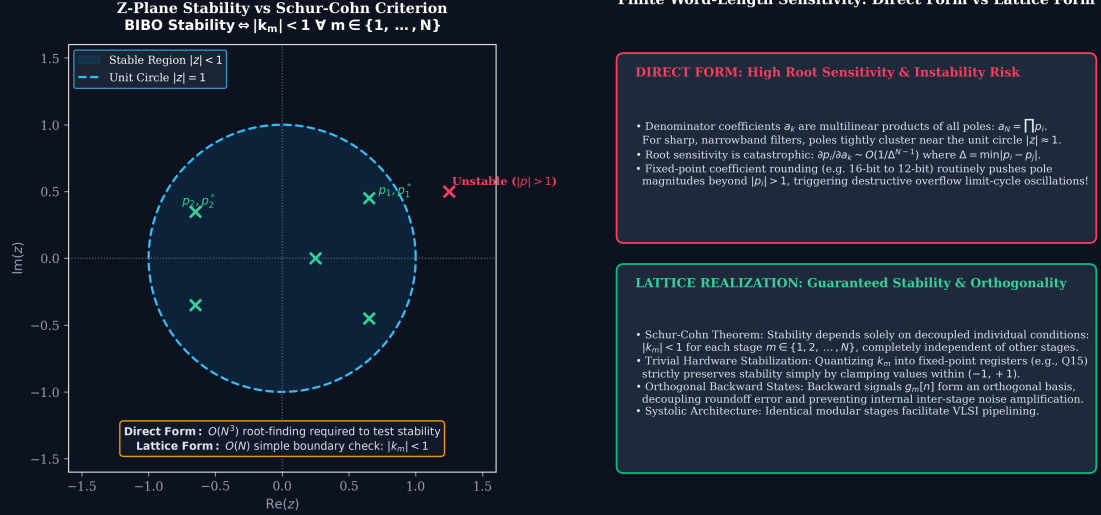


Figure 2: Schur-Cohn Stability Criterion and Finite Word-Length Robustness: (Left) Complex Z -plane showing unit circle stability boundary $|z| = 1$ and the trivial $O(N)$ lattice boundary check ($|k_m| < 1$). (Right) Comparative analysis highlighting the catastrophic root sensitivity of Direct Forms versus the guaranteed stability and orthogonality of Lattice realizations.

The advantages of this theorem in hardware implementation are monumental:

- Decoupled Stability Domain:** In Direct Form, the stable region in coefficient space $(a_1, a_2, \dots, a_N)$ is an irregular, non-convex polytope whose boundaries are complicated nonlinear functions of all coefficients. In Lattice Form, the stable parameter domain is simply the **hypercube** $(-1, +1)^N$. Each coefficient k_m can be verified and bounded completely independently of all other coefficients.
- Guaranteed Stability under Quantization:** When fixed-point DSP hardware truncates or rounds reflection coefficients (e.g., into 16-bit Q15 format), the filter is **guaranteed to remain strictly stable** as long as the quantized value is clamped within $(-1, +1)$. In Direct Form, rounding a single coefficient can easily push high-order clustered poles outside the unit circle ($|p| \geq 1$), triggering destructive self-sustained overflow oscillations.
- Computational Efficiency:** Testing stability via the step-down algorithm requires only $O(N^2)$ additions and multiplications, compared to $O(N^3)$ iterative root-finding.

2.4 General IIR Realization: The Gray-Markel Lattice-Ladder Structure

While the all-pole lattice realizes $H_{ap}(z) = 1/A_N(z)$, practical digital filters require both poles and zeros to achieve sharp cutoffs and notch characteristics:

$$H(z) = \frac{B(z)}{A(z)} = \frac{\sum_{m=0}^M b_m z^{-m}}{1 + \sum_{m=1}^N a_m z^{-m}} \quad (18)$$

Without loss of generality, we assume $M \leq N$. (If $M > N$, polynomial division yields an FIR polynomial plus a strictly proper rational transfer function).

In 1973, A. H. Gray and J. D. Markel invented the **Lattice-Ladder filter structure**, which synthesizes arbitrary numerator zeros without disturbing the all-pole feedback lattice. Their key mathematical insight was that the backward prediction error signals $g_0[n], g_1[n], \dots, g_N[n]$

generated along the lower rail of the all-pole lattice form an **orthogonal basis** for the internal system states:

$$\frac{G_m(z)}{X(z)} = \frac{B_m(z)}{A_N(z)}, \quad m = 0, 1, \dots, N \quad (19)$$

where $B_m(z)$ are the reverse polynomials of $A_m(z)$.

By tapping each backward error signal $g_m[n]$ through a feedforward **ladder multiplier** c_m (often denoted v_m) and accumulating them into a summation bus, the filter output is:

$$y[n] = \sum_{m=0}^N c_m g_m[n] \iff Y(z) = \sum_{m=0}^N c_m G_m(z) \quad (20)$$

Substituting $G_m(z) = \frac{B_m(z)}{A_N(z)} X(z)$:

$$Y(z) = \left(\sum_{m=0}^N c_m \frac{B_m(z)}{A_N(z)} \right) X(z) = \frac{\sum_{m=0}^N c_m B_m(z)}{A_N(z)} X(z) \quad (21)$$

Equating this with $H(z) = B(z)/A_N(z)$, the numerator polynomial is realized as a **linear combination of the reverse polynomials**:

$$\mathbf{B}(z) = \sum_{m=0}^N c_m \mathbf{B}_m(z) = c_0 \mathbf{B}_0(z) + c_1 \mathbf{B}_1(z) + \dots + c_N \mathbf{B}_N(z) \quad (22)$$

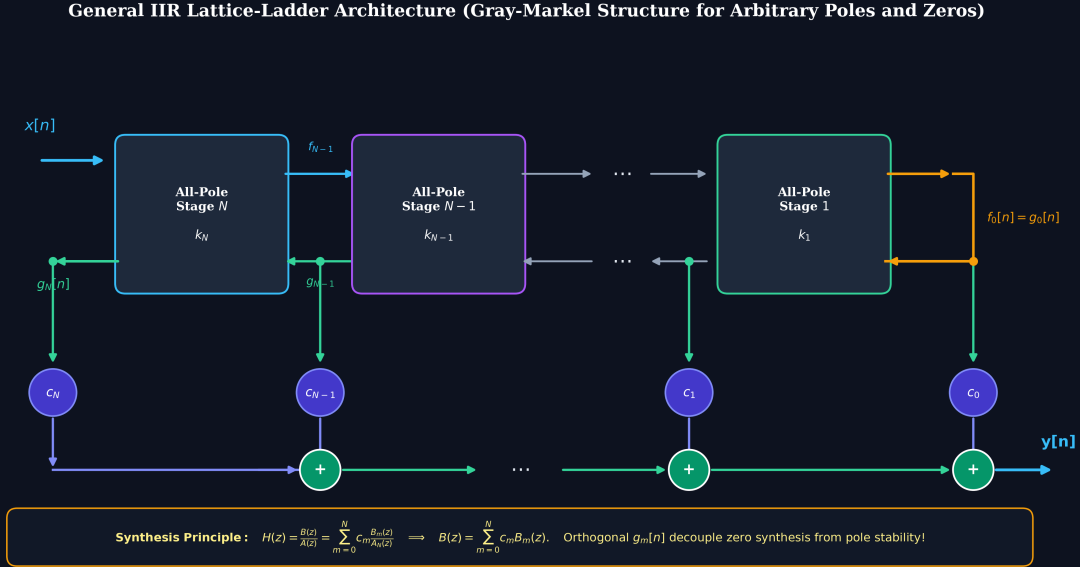


Figure 3: General IIR Lattice-Ladder Realization (Gray-Markel Structure): The all-pole feed-back lattice backbone establishes system poles and generates orthogonal backward error signals $g_m[n]$. Feedforward ladder coefficients c_m tap these signals into an accumulator bus to synthesize arbitrary numerator zeros without altering pole stability.

2.4.1 Derivation of Ladder Coefficients c_m via Reverse Polynomial Peeling

To find the ladder coefficients $c_0, c_1, \dots, c_N$, consider the structure of $B_m(z)$:

$$\begin{aligned} B_0(z) &= 1 \quad (\text{degree } 0) \\ B_1(z) &= k_1 + z^{-1} \quad (\text{degree } 1) \\ B_2(z) &= k_2 + \alpha_2(1)z^{-1} + z^{-2} \quad (\text{degree } 2) \\ &\vdots \\ B_m(z) &= k_m + \alpha_m(m-1)z^{-1} + \dots + z^{-m} \quad (\text{degree } m) \end{aligned} \quad (23)$$

Observe two crucial properties:

1. The coefficient of the highest power z^{-m} in $B_m(z)$ is **identically 1**.
2. For any index $k < m$, the polynomial $B_k(z)$ has degree $k < m$, meaning it contains **no term in z^{-m}** .

Consequently, in the summation $\sum_{m=0}^N c_m B_m(z)$, the highest-order term z^{-N} originates **solely from $c_N B_N(z)$** . Therefore, if the target numerator is $B(z) = \sum_{i=0}^N b_i z^{-i}$:

$$\mathbf{c}_N = \mathbf{b}_N \quad (24)$$

Once c_N is determined, we peel off $c_N B_N(z)$ by defining the reduced numerator polynomial $B^{(N-1)}(z)$:

$$B^{(N-1)}(z) = B(z) - c_N B_N(z) \quad (25)$$

The polynomial $B^{(N-1)}(z)$ is guaranteed to have degree at most $N-1$. Repeating this argument for $m = N-1, N-2, \dots, 1, 0$:

$$\mathbf{c}_m = \text{Coefficient of } \mathbf{z}^{-m} \text{ in } \mathbf{B}^{(m)}(\mathbf{z}) \quad (26)$$

$$\mathbf{B}^{(m-1)}(\mathbf{z}) = \mathbf{B}^{(m)}(\mathbf{z}) - \mathbf{c}_m \mathbf{B}_m(\mathbf{z}) \quad (27)$$

with $B^{(N)}(z) \equiv B(z)$. At the final step $m = 0$, $c_0 = B^{(0)}(z)$, since $B_0(z) = 1$. This recursive projection uniquely and exactly determines all ladder tap weights c_m !

3 3. HARDWARE COMPLEXITY & ARCHITECTURAL AUDIT

To appreciate why lattice-ladder filters are preferred in high-reliability digital signal processors, we perform a rigorous comparative hardware audit against Direct Form I, Direct Form II (Canonical), Cascade Form (Modular Biquads), and Parallel Form.

Table 1: Comprehensive Hardware Resource and Performance Comparison (N -th Order IIR Filter, $M = N$)

Realization Form	Multipliers	Adders	Delays	Sensitivity	Limit Cycles	VLSI Modularity
Direct Form I	$2N + 1$	$2N$	$2N$	Catastrophic	Severe	Poor
Direct Form II	$2N + 1$	$2N$	N	Catastrophic	Severe	Poor
Cascade Form (SOS)	$2N + 1$	$2N$	N	Low	Moderate	High
Parallel Form	$2N + 1$	$2N$	N	Low	Suppressed	High
All-Pole Lattice	$2N$	$2N$	N	Minimum	Zero	Excellent
Lattice-Ladder (IIR)	$3N + 1$	$3N$	N	Minimum	Zero	Systolic

3.1 Key Architectural Trade-offs

1. **Canonical Storage (N Delay Registers):** Although the Lattice-Ladder structure possesses both a feedback lattice and a feedforward ladder, it requires only N **delay registers** z^{-1} , exactly matching the theoretical lower bound for an N -th order system. The states stored in these N registers simultaneously drive the all-pole feedback loop and feed the ladder taps.
2. **Multiplier Overhead ($3N + 1$):** The Lattice-Ladder requires $2N$ reflection multipliers (k_m) plus $N + 1$ ladder multipliers (c_m), totaling $3N + 1$ multipliers. This is higher than Direct Form II ($2N + 1$). However, in silicon implementations, the silicon area spent on multipliers is negligible compared to the massive word-length reductions enabled by lattice robustness:
 - A Direct Form filter often requires 24 to 32 bits of coefficient precision to avoid shifting poles into instability.
 - The Lattice-Ladder filter routinely achieves identical or superior frequency selectivity using only **12 to 16 bits!**
3. **Absence of Zero-Input Overflow Limit Cycles:** By scaling internal nodes using normalized lattice or wave digital filter principles, lattice structures satisfy the **Lyapunov stability criterion**:

$$\sum_{m=0}^N |f_m[n]|^2 + |g_m[n]|^2 \leq \sum_{m=0}^N |f_m[n-1]|^2 + |g_m[n-1]|^2 \quad (28)$$

Energy is strictly dissipative under saturation arithmetic, completely eliminating parasitic autonomous limit-cycle oscillations.

4. **Systolic VLSI Pipelining:** Unlike Direct Form structures which feature global feedback accumulation rails of length N , lattice stages communicate solely with nearest neighbors. This local communication topology makes lattice filters ideal for high-throughput **systolic array pipelining**.

4. STEP-BY-STEP WORKED NUMERICAL EXAMPLES

4.1 Example 20.1: All-Pole Lattice Step-Down Analysis & Stability Verification

Problem Statement: An all-pole IIR digital filter is specified by the denominator polynomial:

$$A_3(z) = 1 - 0.9z^{-1} + 0.64z^{-2} - 0.5z^{-3} \quad (29)$$

Determine the reflection coefficients k_1, k_2, k_3 , evaluate filter stability using the Schur-Cohn criterion, and write the state difference equations for all lattice stages.

Step 1: Stage 3 Analysis ($m = 3$) From $A_3(z)$, the direct form coefficients are:

$$\alpha_3(1) = -0.9, \quad \alpha_3(2) = 0.64, \quad \alpha_3(3) = -0.5 \quad (30)$$

The reflection coefficient is:

$$\mathbf{k}_3 = \alpha_3(3) = -0.5 \quad (31)$$

Compute the denominator factor:

$$1 - k_3^2 = 1 - (-0.5)^2 = 1 - 0.25 = 0.75 \quad (32)$$

Apply the step-down formula (16) to compute $\alpha_2(1)$ and $\alpha_2(2)$:

$$\alpha_2(1) = \frac{\alpha_3(1) - k_3\alpha_3(2)}{1 - k_3^2} = \frac{-0.9 - (-0.5)(0.64)}{0.75} = \frac{-0.9 + 0.32}{0.75} = \frac{-0.58}{0.75} \approx -\mathbf{0.7733} \quad (33)$$

$$\alpha_2(2) = \frac{\alpha_3(2) - k_3\alpha_3(1)}{1 - k_3^2} = \frac{0.64 - (-0.5)(-0.9)}{0.75} = \frac{0.64 - 0.45}{0.75} = \frac{0.19}{0.75} \approx \mathbf{0.2533} \quad (34)$$

The reduced second-order polynomial is:

$$A_2(z) = 1 - 0.7733z^{-1} + 0.2533z^{-2} \quad (35)$$

Step 2: Stage 2 Analysis ($m = 2$) From $A_2(z)$, the reflection coefficient is:

$$\mathbf{k_2 = \alpha_2(2) = 0.2533} \quad (36)$$

Compute the denominator factor:

$$1 - k_2^2 = 1 - (0.2533)^2 = 1 - 0.06418 = 0.93582 \quad (37)$$

Apply the step-down formula with $m = 2$ and $i = 1$ to compute $\alpha_1(1)$. Note that the complementary reverse index is $m - i = 2 - 1 = \mathbf{1}$, so $\alpha_2(m - i) = \alpha_2(2 - 1) = \alpha_2(\mathbf{1})$:

$$\alpha_1(1) = \frac{\alpha_2(1) - k_2\alpha_2(2 - 1)}{1 - k_2^2} = \frac{\alpha_2(1)(1 - k_2)}{(1 - k_2)(1 + k_2)} = \frac{\alpha_2(1)}{1 + k_2} \quad (38)$$

$$\alpha_1(1) = \frac{-0.77333}{1 + 0.25333} = \frac{-0.77333}{1.25333} \approx -\mathbf{0.6170} \quad (39)$$

The reduced first-order polynomial is:

$$A_1(z) = 1 - 0.6170z^{-1} \quad (40)$$

Step 3: Stage 1 Analysis ($m = 1$) From $A_1(z)$, the reflection coefficient is:

$$\mathbf{k_1 = \alpha_1(1) = -0.6170} \quad (41)$$

Step 4: Schur-Cohn Stability Verification We examine the magnitudes of all reflection coefficients:

$$|k_1| = |-0.6170| = 0.6170 < 1 \quad (42)$$

$$|k_2| = |0.2533| = 0.2533 < 1 \quad (43)$$

$$|k_3| = |-0.5000| = 0.5000 < 1 \quad (44)$$

Because $|k_m| < 1$ for all $m \in \{1, 2, 3\}$, the Schur-Cohn criterion is strictly satisfied. **The filter is unconditionally BIBO stable**, with all 3 poles residing strictly inside the unit circle $|z| < 1$.

Step 5: Lattice State Difference Equations With $x[n] = f_3[n]$, the real-time sample-by-sample update equations are:

$$f_2[n] = f_3[n] - k_3g_2[n - 1] = x[n] - (-0.5)g_2[n - 1] = x[n] + 0.5g_2[n - 1] \quad (45)$$

$$g_3[n] = k_3f_2[n] + g_2[n - 1] = -0.5f_2[n] + g_2[n - 1] \quad (46)$$

$$f_1[n] = f_2[n] - k_2g_1[n - 1] = f_2[n] - 0.2533g_1[n - 1] \quad (47)$$

$$g_2[n] = k_2f_1[n] + g_1[n - 1] = 0.2533f_1[n] + g_1[n - 1] \quad (48)$$

$$f_0[n] = f_1[n] - k_1g_0[n - 1] = f_1[n] + 0.6170g_0[n - 1] \quad (49)$$

$$g_1[n] = k_1f_0[n] + g_0[n - 1] = -0.6170f_0[n] + g_0[n - 1] \quad (50)$$

$$y[n] = f_0[n] = g_0[n] \quad (51)$$

4.2 Example 20.2: General Pole-Zero IIR Lattice-Ladder Synthesis

Problem Statement: Synthesize the complete Lattice-Ladder filter for the 2nd-order rational IIR transfer function:

$$H(z) = \frac{B(z)}{A(z)} = \frac{2 + 3z^{-1} + 4z^{-2}}{1 - 0.9z^{-1} + 0.2z^{-2}} \quad (52)$$

Find the reflection coefficients k_1, k_2 , compute the reverse polynomials $B_m(z)$, calculate the feedforward ladder coefficients c_0, c_1, c_2 , and algebraically verify the reconstruction of $H(z)$.

Step 1: Denominator Step-Down Analysis ($A_2(z) \rightarrow k_m$) From $A_2(z) = 1 - 0.9z^{-1} + 0.2z^{-2}$:

$$\alpha_2(1) = -0.9, \quad \alpha_2(2) = 0.2 \quad (53)$$

The stage 2 reflection coefficient is:

$$\mathbf{k}_2 = \alpha_2(2) = \mathbf{0.2} \quad (54)$$

Compute the denominator divisor:

$$1 - k_2^2 = 1 - (0.2)^2 = 1 - 0.04 = 0.96 \quad (55)$$

Apply the step-down formula (16) with $m = 2$ and $i = 1$ to find $\alpha_1(1)$. Note that the complementary reverse index is $m - i = 2 - 1 = 1$, so $\alpha_2(m - i) = \alpha_2(2 - 1) = \alpha_2(1) = -\mathbf{0.9}$:

$$\begin{aligned} \alpha_1(1) &= \frac{\alpha_2(1) - k_2 \alpha_2(2 - 1)}{1 - k_2^2} = \frac{\alpha_2(1) - k_2 \alpha_2(1)}{1 - k_2^2} \\ &= \frac{-0.9 - 0.2(-0.9)}{0.96} = \frac{-0.9(1 - 0.2)}{0.96} = \frac{-0.72}{0.96} = -\mathbf{0.75} \end{aligned} \quad (56)$$

The reduced first-order denominator polynomial is:

$$A_1(z) = 1 + \alpha_1(1)z^{-1} = \mathbf{1 - 0.75z^{-1}} \quad (57)$$

(Equivalently obtained via polynomial step-down: $A_1(z) = \frac{A_2(z) - k_2 B_2(z)}{1 - k_2^2}$). The stage 1 reflection coefficient is:

$$\mathbf{k}_1 = \alpha_1(1) = -\mathbf{0.75} \quad (58)$$

Stability Check: $|k_1| = |-0.75| = 0.75 < 1$ and $|k_2| = |0.20| = 0.20 < 1$. By the Schur-Cohn criterion, the system is strictly BIBO stable (poles lie at $z = 0.5$ and $z = 0.4$, well inside the unit circle).

Step 2: Construct the Reverse Polynomials $B_m(z)$

Using the definition $B_m(z) = z^{-m} A_m(z^{-1})$:

$$A_0(z) = 1 \implies \mathbf{B_0(z) = 1} \quad (59)$$

$$A_1(z) = 1 - 0.75z^{-1} \implies B_1(z) = z^{-1}(1 - 0.75z) = -\mathbf{0.75} + \mathbf{z^{-1}} \quad (60)$$

$$A_2(z) = 1 - 0.9z^{-1} + 0.2z^{-2} \implies B_2(z) = z^{-2}(1 - 0.9z + 0.2z^2) = \mathbf{0.2 - 0.9z^{-1} + z^{-2}} \quad (61)$$

Step 3: Extract Ladder Coefficients c_m **via Polynomial Peeling** The target numerator is $B(z) = B^{(2)}(z) = 2 + 3z^{-1} + 4z^{-2}$.

- **Stage 2 Ladder Tap** (c_2): The coefficient of z^{-2} in $B^{(2)}(z)$ is 4. Since the coefficient of z^{-2} in $B_2(z)$ is 1:

$$\mathbf{c_2 = 4.0} \quad (62)$$

Peel off $c_2B_2(z)$:

$$c_2B_2(z) = 4.0(0.2 - 0.9z^{-1} + z^{-2}) = 0.8 - 3.6z^{-1} + 4.0z^{-2} \quad (63)$$

$$\begin{aligned} B^{(1)}(z) &= B^{(2)}(z) - c_2B_2(z) \\ &= (2 + 3z^{-1} + 4z^{-2}) - (0.8 - 3.6z^{-1} + 4.0z^{-2}) \\ &= (2 - 0.8) + (3 - (-3.6))z^{-1} + (4 - 4)z^{-2} \\ &= \mathbf{1.2} + \mathbf{6.6z^{-1}} \end{aligned} \quad (64)$$

- **Stage 1 Ladder Tap (c_1):** The coefficient of z^{-1} in $B^{(1)}(z)$ is 6.6. Since the coefficient of z^{-1} in $B_1(z)$ is 1:

$$\mathbf{c_1 = 6.6} \quad (65)$$

Peel off $c_1B_1(z)$:

$$c_1B_1(z) = 6.6(-0.75 + z^{-1}) = -4.95 + 6.6z^{-1} \quad (66)$$

$$\begin{aligned} B^{(0)}(z) &= B^{(1)}(z) - c_1B_1(z) \\ &= (1.2 + 6.6z^{-1}) - (-4.95 + 6.6z^{-1}) \\ &= (1.2 - (-4.95)) + (6.6 - 6.6)z^{-1} \\ &= \mathbf{6.15} \end{aligned} \quad (67)$$

- **Stage 0 Ladder Tap (c_0):** Since $B_0(z) = 1$, the scalar constant $B^{(0)}(z)$ directly provides c_0 :

$$\mathbf{c_0 = 6.15} \quad (68)$$

Step 4: Algebraic Verification of Numerator Synthesis We verify that the linear combination $\sum_{m=0}^2 c_m B_m(z)$ reconstructs $B(z)$ exactly:

$$\begin{aligned} \sum_{m=0}^2 c_m B_m(z) &= c_0 B_0(z) + c_1 B_1(z) + c_2 B_2(z) \\ &= 6.15(1) + 6.6(-0.75 + z^{-1}) + 4.0(0.2 - 0.9z^{-1} + z^{-2}) \\ &= 6.15 - 4.95 + 6.6z^{-1} + 0.8 - 3.6z^{-1} + 4.0z^{-2} \\ &= (6.15 - 4.95 + 0.8) + (6.6 - 3.6)z^{-1} + 4.0z^{-2} \\ &= \mathbf{2 + 3z^{-1} + 4z^{-2}} \equiv B(z) \quad (\mathbf{Q.E.D.}) \end{aligned} \quad (69)$$

The synthesized Lattice-Ladder filter matches the exact specification!

Worked Example 20.2: Complete Synthesized 2nd-Order IIR Lattice-Ladder Filter
 $H(z) = \frac{2 + 3z^{-1} + 4z^{-2}}{1 - 0.9z^{-1} + 0.2z^{-2}} \Rightarrow k_1 = -0.75, k_2 = 0.2$ and $c_0 = 6.15, c_1 = 6.6, c_2 = 4.0$

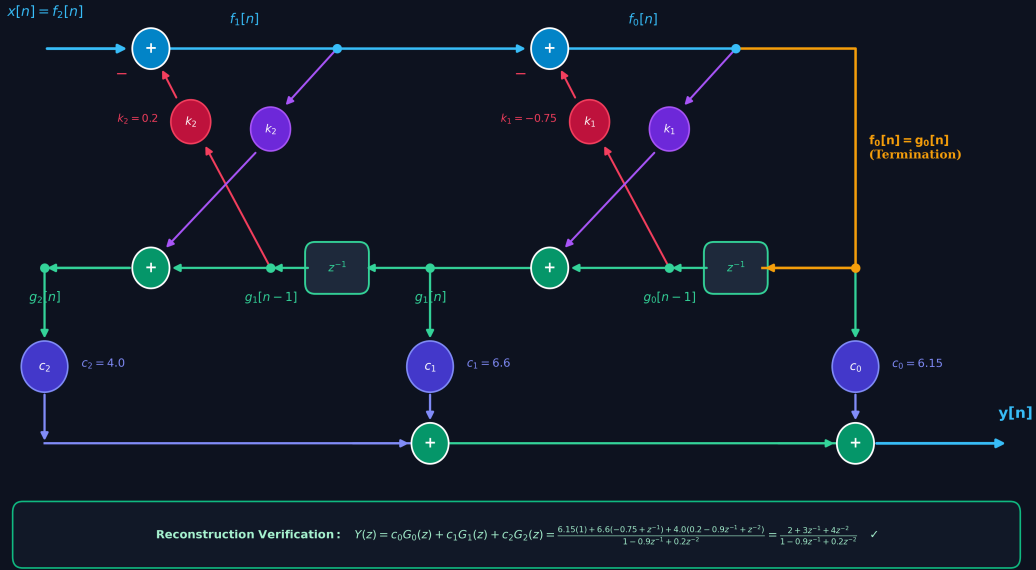


Figure 4: Synthesized 2nd-Order Lattice-Ladder Architecture for Worked Example 20.2: Denominator reflection coefficients $k_1 = -0.75$, $k_2 = 0.20$ and feedforward ladder tap weights $c_0 = 6.15$, $c_1 = 6.6$, $c_2 = 4.0$. Note the canonical use of only 2 delay registers z^{-1} .

4.3 Example 20.3: Reverse Analysis (Synthesis from Lattice Parameters)

Problem Statement: A 2-stage Lattice-Ladder filter is implemented in DSP hardware with reflection coefficients:

$$k_1 = 0.5, \quad k_2 = -0.4 \quad (70)$$

and feedforward ladder coefficients:

$$c_0 = 2, \quad c_1 = 3, \quad c_2 = 1 \quad (71)$$

Determine the rational transfer function $H(z) = B(z)/A(z)$ of the filter.

Step 1: Step-Up Recursion for Denominator Polynomial $A_2(z)$ Starting with initial conditions $A_0(z) = 1$ and $B_0(z) = 1$:

- **Stage 1 ($m = 1$):**

$$A_1(z) = A_0(z) + k_1 z^{-1} B_0(z) = 1 + 0.5z^{-1} \quad (72)$$

$$B_1(z) = k_1 A_0(z) + z^{-1} B_0(z) = 0.5 + z^{-1} \quad (73)$$

- **Stage 2 ($m = 2$):**

$$\begin{aligned} A_2(z) &= A_1(z) + k_2 z^{-1} B_1(z) \\ &= (1 + 0.5z^{-1}) + (-0.4)z^{-1}(0.5 + z^{-1}) \\ &= 1 + 0.5z^{-1} - 0.2z^{-1} - 0.4z^{-2} \\ &= \mathbf{1 + 0.3z^{-1} - 0.4z^{-2}} \end{aligned} \quad (74)$$

Compute $B_2(z)$ for the numerator summation:

$$\begin{aligned}
B_2(z) &= k_2 A_1(z) + z^{-1} B_1(z) \\
&= -0.4(1 + 0.5z^{-1}) + z^{-1}(0.5 + z^{-1}) \\
&= -0.4 - 0.2z^{-1} + 0.5z^{-1} + z^{-2} \\
&= -\mathbf{0.4} + \mathbf{0.3z^{-1}} + \mathbf{z^{-2}}
\end{aligned} \tag{75}$$

Step 2: Numerator Polynomial Synthesis Using the ladder summation formula (22):

$$\begin{aligned}
B(z) &= c_0 B_0(z) + c_1 B_1(z) + c_2 B_2(z) \\
&= 2(1) + 3(0.5 + z^{-1}) + 1(-0.4 + 0.3z^{-1} + z^{-2}) \\
&= 2 + 1.5 + 3z^{-1} - 0.4 + 0.3z^{-1} + z^{-2} \\
&= (2 + 1.5 - 0.4) + (3 + 0.3)z^{-1} + z^{-2} \\
&= \mathbf{3.1} + \mathbf{3.3z^{-1}} + \mathbf{z^{-2}}
\end{aligned} \tag{76}$$

Step 3: Overall Transfer Function The complete rational IIR transfer function realized by the filter is:

$$\mathbf{H(z)} = \frac{\mathbf{3.1} + \mathbf{3.3z^{-1}} + \mathbf{z^{-2}}}{\mathbf{1} + \mathbf{0.3z^{-1}} - \mathbf{0.4z^{-2}}} \tag{77}$$

Stability Verification: Denominator roots are solutions to $z^2 + 0.3z - 0.4 = 0 \implies (z + 0.8)(z - 0.5) = 0$. The poles are at $z = -0.8$ and $z = 0.5$. Since both $|-0.8| < 1$ and $|0.5| < 1$, the filter is confirmed to be stable, consistent with $|k_1| = 0.5 < 1$ and $|k_2| = 0.4 < 1$.

5. FACULTY INSIGHTS, COMMON PITFALLS & EXAM TRAPS

- **Pitfall 1: Confusing FIR vs All-Pole IIR Signal Flow Direction:** In an FIR lattice filter, the input $x[n]$ enters at stage 0 ($f_0 = g_0 = x[n]$) and signal flows left-to-right toward stage N ($y[n] = f_N[n]$). In an All-Pole IIR lattice, **the input $x[n]$ enters at stage N** ($f_N[n] = x[n]$) and propagates through internal feedback loops toward stage 0 ($y[n] = f_0[n] = g_0[n]$). Students who reverse this flow receive zero marks on exam block diagrams.
- **Pitfall 2: Sign of the Feedback Term (f_{m-1} equation):** In an FIR lattice, $f_m[n] = f_{m-1}[n] + k_m g_{m-1}[n-1]$ (addition). When inverting the flow for an all-pole IIR lattice, solving for $f_{m-1}[n]$ introduces a **crucial minus sign**:

$$f_{m-1}[n] = f_m[n] - k_m g_{m-1}[n-1] \tag{78}$$

Forgetting this negative sign turns negative feedback into unstable positive feedback!

- **Pitfall 3: Tapping Forward vs Backward Residuals in the Ladder:** In the Lattice-Ladder structure, the ladder tap weights c_m **MUST** multiply the **backward prediction error signals** $g_m[n]$, NOT the forward error signals $f_m[n]$. It is the backward signals $g_m[n]$ whose transfer functions equal $B_m(z)/A_N(z)$, providing the orthogonal polynomial basis.
- **Pitfall 4: What Happens When $|k_m| = 1$ in the Step-Down Algorithm?** If during the step-down recursion any $|k_m| = 1$, the divisor $1 - k_m^2 = 0$, causing division by zero!

This occurs if and only if the polynomial $A_m(z)$ has roots located **exactly on the unit circle** $|z| = 1$. In that case, the filter is marginally stable (oscillatory), and standard step-down recursion terminates. If $|k_m| > 1$, the filter has at least one pole strictly outside the unit circle and is **unstable**.

- **Pitfall 5: Order of Ladder Weight Extraction:** Ladder coefficients must be extracted starting from the **highest order** c_N **down to** c_0 . You cannot compute c_0 first, because $B_N(z), B_{N-1}(z), \dots, B_1(z)$ all contribute constant terms to the zeroth power z^0 . Only the highest power z^{-m} has a contribution solely from $c_m B_m(z)$.

6 6. QUICK REVISION & SUMMARY CHEAT-SHEET

Table 2: Comprehensive IIR Lattice Realization Quick Reference

Concept / Parameter	Mathematical Definition / Operational Formula
All-Pole Lattice Forward Eq.	$f_{m-1}[n] = f_m[n] - k_m g_{m-1}[n-1]$
All-Pole Lattice Backward Eq.	$g_m[n] = k_m f_{m-1}[n] + g_{m-1}[n-1]$
Boundary Conditions	$f_N[n] = x[n], \quad y[n] = f_0[n] = g_0[n]$
Step-Down Reflection Coeff.	$k_m = \alpha_m(m)$
Step-Down Polynomial Recursion	$\alpha_{m-1}(i) = \frac{\alpha_m(i) - k_m \alpha_m(m-i)}{1 - k_m^2}, \quad i = 1, \dots, m-1$
Reverse Polynomial Relation	$B_m(z) = z^{-m} A_m(z^{-1})$
Schur-Cohn Stability Rule	BIBO Stable $\iff k_m < 1 \quad \forall m \in \{1, 2, \dots, N\}$
Lattice-Ladder Numerator Sum	$B(z) = \sum_{m=0}^N c_m B_m(z)$
Ladder Weight Peeling	$c_m = \text{Coeff of } z^{-m} \text{ in } B^{(m)}(z)$
Reduced Polynomial Update	$B^{(m-1)}(z) = B^{(m)}(z) - c_m B_m(z)$
Delay Canonicity	Canonical: Exactly N registers for order N
Limit Cycle Behavior	Autonomous overflow limit cycles completely suppressed

APPENDIX: VISUAL ARCHITECTURAL ATLAS

A.1 The Inversion Duality: FIR Forward Lattice vs. All-Pole IIR Feedback Lattice

The fundamental foundation of IIR lattice realization is the mathematical duality between forward FIR prediction and feedback all-pole filtering. Figure 5 provides a direct, node-by-node visual comparison between a single forward FIR stage and the corresponding inverted all-pole IIR stage.

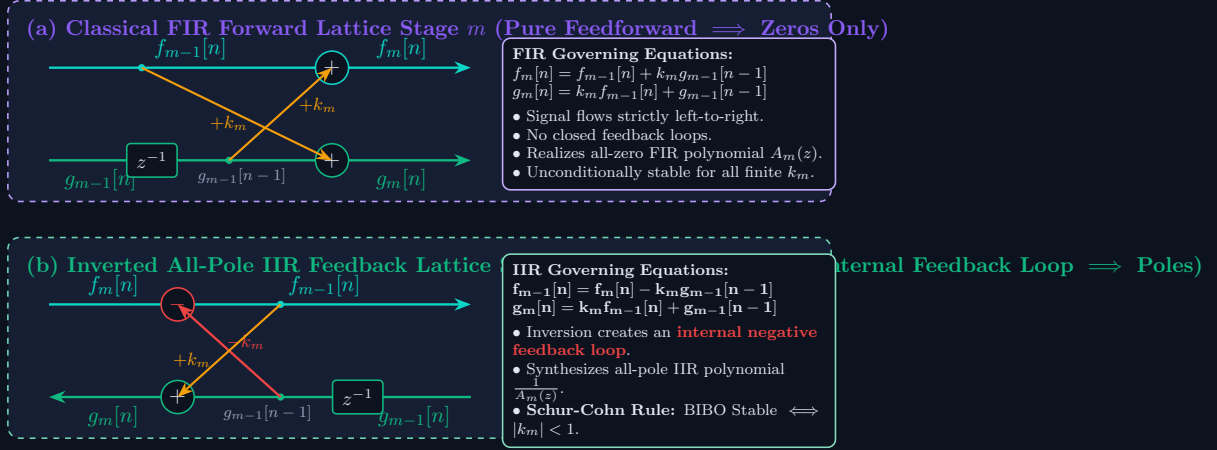


Figure 5: Architectural Inversion Duality: (a) Forward FIR lattice generating zeros without feedback. (b) Inverted All-Pole IIR lattice where algebraic inversion isolates $f_{m-1}[n]$, introducing a negative feedback loop that synthesizes system poles with decoupled stability bounds.

A.2 Complete Signal Propagation in Cascaded All-Pole IIR Lattice ($N = 3$)

Figure 6 illustrates how N inverted lattice stages cascade to form an N -th order all-pole filter. The input signal $x[n]$ is injected at Stage N ($f_N[n] = x[n]$), traverses through negative feedback loops, and emerges as the output at Stage 0 ($y[n] = f_0[n] = g_0[n]$).

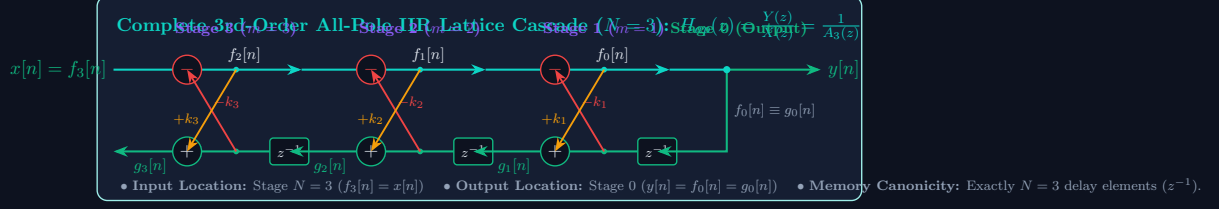


Figure 6: Complete Cascaded 3rd-Order All-Pole IIR Lattice Realization ($N = 3$): Signal traverses top rail from left to right through local feedback subtraction stages, loops through Stage 0 to produce output $y[n]$, and propagates backward error sequence $g_m[n]$ through the delay spine.

A.3 General IIR Synthesis: The Gray-Markel Lattice-Ladder (Poles + Zeros)

Figure 7 depicts the canonical Gray-Markel Lattice-Ladder structure for general pole-zero IIR filters. The all-pole feedback lattice forms the structural backbone establishing system poles, while feedforward ladder taps c_m weight the orthogonal backward prediction errors $g_m[n]$ to synthesize numerator zeros.

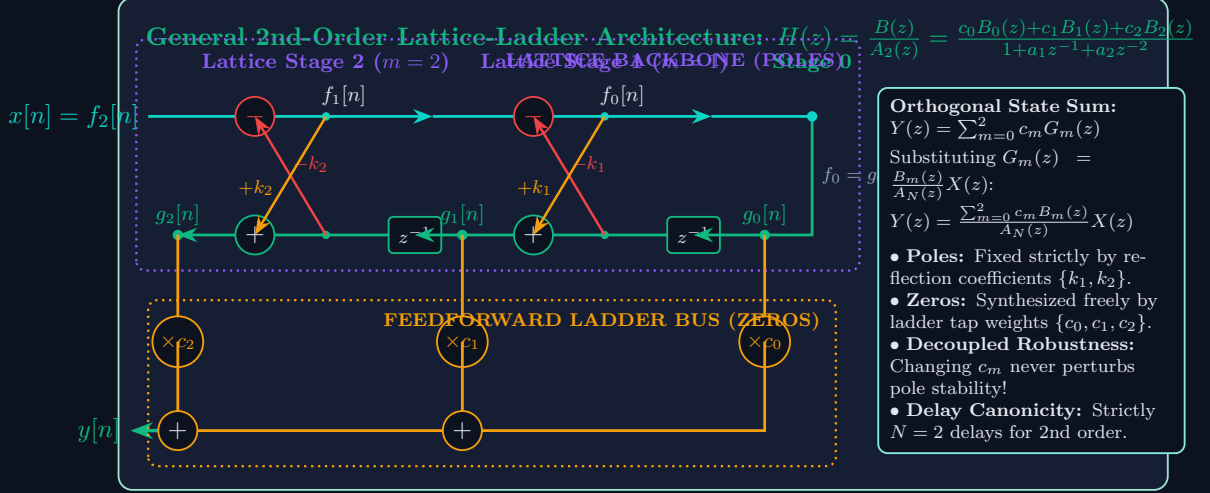


Figure 7: Canonical Gray-Markel Lattice-Ladder Filter ($N = 2$): The upper feedback lattice establishes system poles with Schur-Cohn stability ($|k_m| < 1$). The lower feedforward ladder taps orthogonal states $g_m[n]$ to synthesize numerator zeros via linear combination $\sum c_m B_m(z)$ without altering stability.

A.4 Algorithmic Synthesis Flowcharts

This section presents the complete, self-contained operational flowcharts for the two synthesis algorithms of the Gray-Markel Lattice-Ladder structure:

1. **Figure 8: The Step-Down (Levinson-Durbin Reverse) Algorithm**, extracting reflection coefficients k_m with concurrent $O(N)$ Schur-Cohn stability verification.
2. **Figure 9: The Reverse Polynomial Peeling Algorithm**, projecting target numerator $B(z)$ onto the orthogonal reverse basis to extract ladder tap weights c_m .

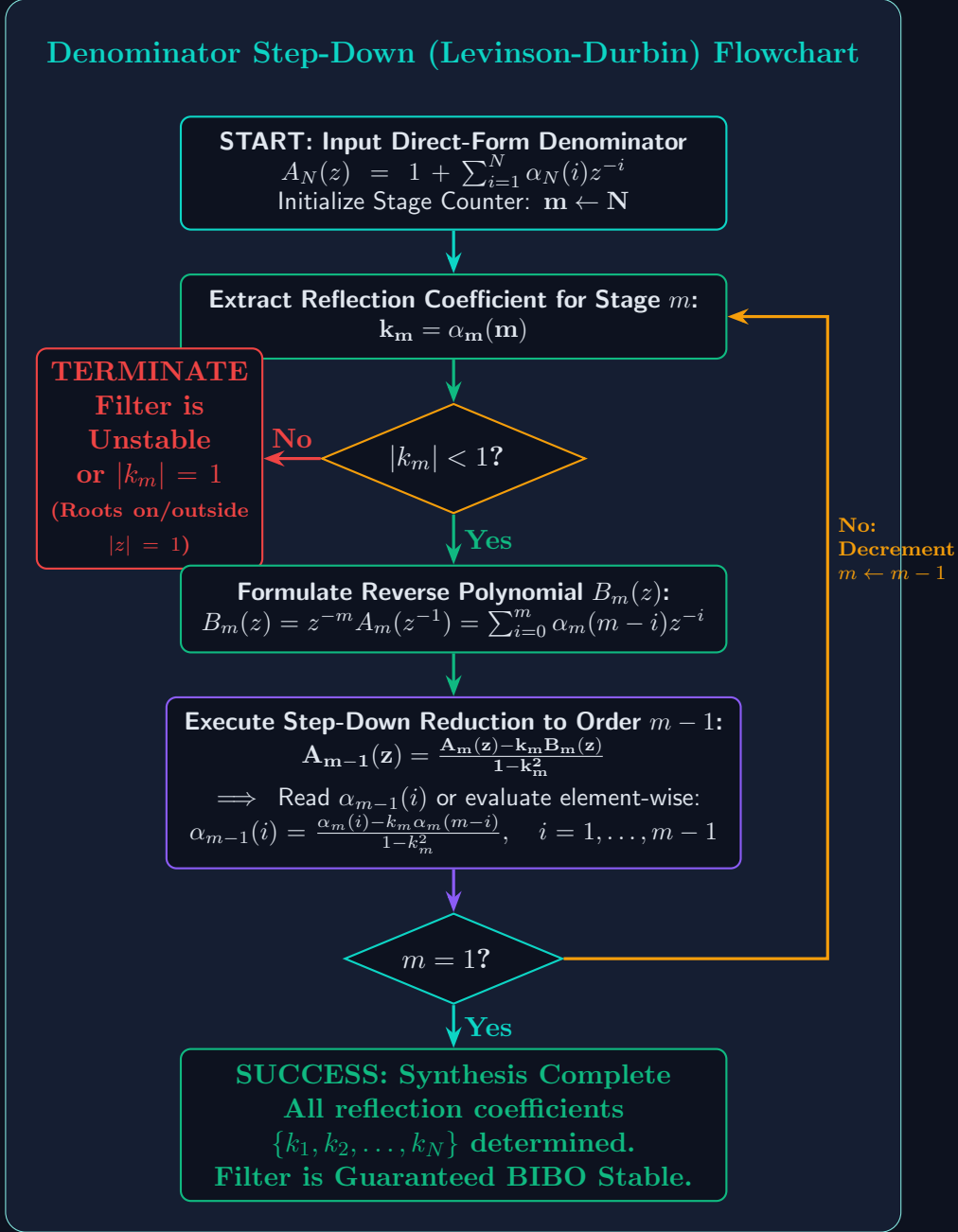


Figure 8: Denominator Step-Down (Levinson-Durbin Reverse) Flowchart: Systematic extraction of reflection coefficients k_m with concurrent $O(N)$ Schur-Cohn stability verification at every stage.

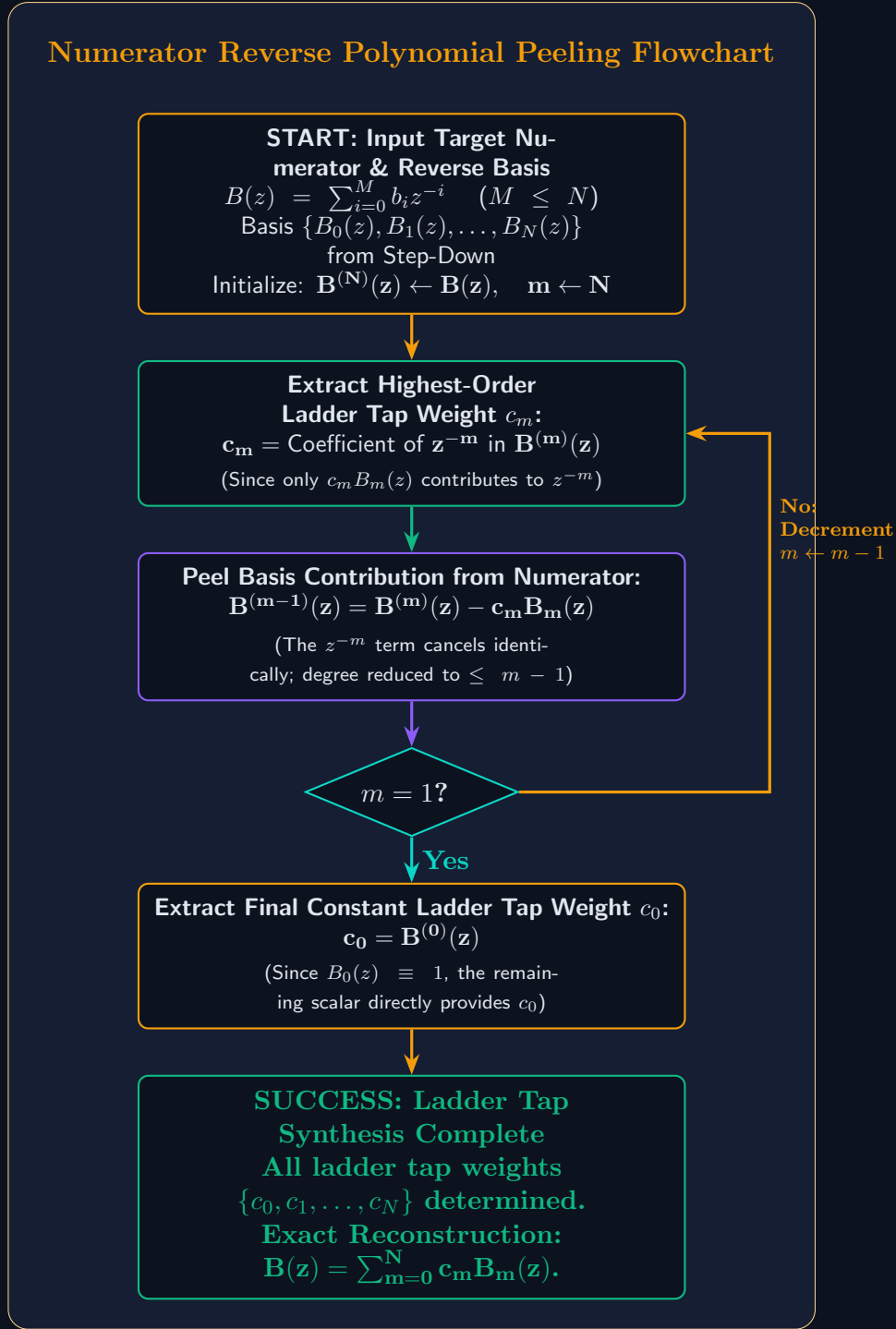


Figure 9: Numerator Reverse Polynomial Peeling Flowchart: Systematic projection of target numerator $B(z)$ onto the orthogonal basis $B_m(z)$ to extract ladder tap weights c_m .

A.5 Step-by-Step Flowchart Execution Trace (Worked 2-Iteration Example)

To demonstrate how both flowcharts execute sample-by-sample in practice, we trace the complete 2nd-order transfer function from **Example 20.2**:

$$H(z) = \frac{B(z)}{A_2(z)} = \frac{2 + 3z^{-1} + 4z^{-2}}{1 - 0.9z^{-1} + 0.2z^{-2}} \quad (79)$$

Because the filter order is $N = 2$, each flowchart executes for **exactly two iterations** ($m = 2$ and $m = 1$).

Trace 1: Denominator Step-Down Flowchart (Figure 8)

- **START Block:** Input denominator polynomial: $A_2(z) = 1 - 0.9z^{-1} + 0.2z^{-2}$. Coefficients: $\alpha_2(1) = -0.9$, $\alpha_2(2) = 0.2$. Set initial counter: $\mathbf{m} = 2$.
- **Iteration 1 ($m = 2$):**

1. Block "Extract k_m ":

$$\mathbf{k}_2 = \alpha_2(2) = \mathbf{0.20}$$

2. Decision Diamond " $|k_m| < 1$?: $|k_2| = |0.20| = 0.20 < 1 \implies$ **YES (Passes Schur-Cohn stability check).**

3. Block "Formulate Reverse Polynomial $B_m(z)$ ":

$$B_2(z) = z^{-2}A_2(z^{-1}) = \mathbf{0.20 - 0.90z^{-1} + 1.00z^{-2}}$$

4. Block "Execute Polynomial Step-Down Reduction":

Compute denominator factor: $1 - k_2^2 = 1 - (0.20)^2 = 1 - 0.04 = \mathbf{0.96}$.

Direct Polynomial Reduction (as ordered in flowchart): Using $A_2(z) = 1 - 0.90z^{-1} + 0.20z^{-2}$ and $B_2(z) = 0.20 - 0.90z^{-1} + 1.00z^{-2}$:

$$\begin{aligned} A_1(z) &= \frac{A_2(z) - k_2 B_2(z)}{1 - k_2^2} \\ &= \frac{(1 - 0.90z^{-1} + 0.20z^{-2}) - 0.20(0.20 - 0.90z^{-1} + 1.00z^{-2})}{0.96} \\ &= \frac{(1 - 0.04) + (-0.90 - (-0.18))z^{-1} + (0.20 - 0.20)z^{-2}}{0.96} \\ &= \frac{0.96 - 0.72z^{-1} + 0z^{-2}}{0.96} = \mathbf{1 - 0.75z^{-1}} \end{aligned} \quad (80)$$

The highest-degree term $(0.20 - 0.20)z^{-2} \equiv 0$ cancels identically, reducing the order to 1.

Identify Reduced Polynomial Coefficient α_{m-1} : From the reduced polynomial $A_1(z) = 1 + \alpha_1(1)z^{-1} = 1 - 0.75z^{-1}$, we directly read off:

$$\alpha_1(1) = \mathbf{-0.75}$$

Cross-Verification via Element-wise Coefficient Formula:

Applying the scalar formula $\alpha_{m-1}(i) = \frac{\alpha_m(i) - k_m \alpha_m(m-i)}{1 - k_m^2}$ for $m = 2, i = 1$: Note carefully that the reverse index is $m - i = 2 - 1 = \mathbf{1}$, so $\alpha_m(m - i) = \alpha_2(2 - 1) = \alpha_2(1) = \mathbf{-0.90}$ (and NOT $\alpha_2(2) = 0.20$):

$$\begin{aligned} \alpha_1(1) &= \frac{\alpha_2(1) - k_2 \alpha_2(2 - 1)}{1 - k_2^2} = \frac{\alpha_2(1) - k_2 \alpha_2(1)}{1 - k_2^2} \\ &= \frac{-0.90 - (0.20)(-0.90)}{0.96} = \frac{-0.90 + 0.18}{0.96} = \frac{-0.72}{0.96} = \mathbf{-0.75} \end{aligned} \quad (81)$$

Both polynomial reduction and scalar coefficient recursion yield identical results:
 $\mathbf{A}_1(z) = 1 - 0.75z^{-1}$ and $\alpha_1(1) = -0.75$.

5. *Decision Diamond "m = 1?"*: $m = 2 \neq 1 \implies \mathbf{NO}$.

6. *Loopback Branch*: Decrement $m \leftarrow m - 1 = 2 - 1 = 1$. Loop back to top!

• **Iteration 2 ($m = 1$):**

1. *Block "Extract k_m "*: From the active polynomial $A_1(z) = 1 - 0.75z^{-1}$:

$$\mathbf{k}_1 = \alpha_1(1) = -0.75$$

2. *Decision Diamond " $|k_m| < 1$?"*: $|k_1| = |-0.75| = 0.75 < 1 \implies \mathbf{YES}$ (**Passes Schur-Cohn stability check**).

3. *Block "Formulate Reverse Polynomial $B_m(z)$ "*:

$$B_1(z) = z^{-1}A_1(z^{-1}) = -0.75 + 1.00z^{-1}$$

4. *Block "Execute Polynomial Step-Down Reduction"*:

Computing the reduction to Stage 0:

$$\begin{aligned} A_0(z) &= \frac{A_1(z) - k_1 B_1(z)}{1 - k_1^2} \\ &= \frac{(1 - 0.75z^{-1}) - (-0.75)(-0.75 + 1.00z^{-1})}{1 - (-0.75)^2} \\ &= \frac{(1 - 0.5625) + (-0.75 + 0.75)z^{-1}}{0.4375} = \frac{0.4375}{0.4375} = 1.00 \equiv \mathbf{A}_0(z) \end{aligned} \quad (82)$$

which confirms boundary condition $A_0(z) = 1$ and $B_0(z) = 1$.

5. *Decision Diamond "m = 1?"*: $m = 1 \implies \mathbf{YES}$.

6. *TERMINAL SUCCESS Block*: Algorithm terminates successfully.

$$\{\mathbf{k}_1 = -0.75, \quad \mathbf{k}_2 = 0.20\}$$

Because $|k_1| < 1$ and $|k_2| < 1$, the filter is **strictly BIBO stable**!

Trace 2: Numerator Polynomial Peeling Flowchart (Figure 9)

- **START Block:** Input target numerator: $B^{(2)}(z) = B(z) = 2 + 3z^{-1} + 4z^{-2}$. Basis polynomials from Step-Down:

$$B_0(z) = 1, \quad B_1(z) = -0.75 + z^{-1}, \quad B_2(z) = 0.2 - 0.9z^{-1} + z^{-2}$$

Initialize counter: $\mathbf{m} = 2$.

- **Iteration 1 ($m = 2$):**

1. *Block "Extract c_m ":* The coefficient of the highest power z^{-2} in $B^{(2)}(z)$ is 4.0.

$$\mathbf{c_2 = 4.00}$$

2. *Block "Peel Basis Contribution":* Compute $c_2 B_2(z) = 4.0(0.20 - 0.90z^{-1} + 1.00z^{-2}) = 0.80 - 3.60z^{-1} + 4.00z^{-2}$.

$$\begin{aligned} B^{(1)}(z) &= B^{(2)}(z) - c_2 B_2(z) \\ &= (2 - 0.80) + (3 - (-3.60))z^{-1} + (4 - 4)z^{-2} = \mathbf{1.20 + 6.60z^{-1}} \end{aligned} \quad (83)$$

(Observe that z^{-2} is canceled identically, leaving a 1st-order polynomial).

3. *Decision Diamond "m = 1?":* $m = 2 \neq 1 \implies \mathbf{NO}$.
4. *Loopback Branch:* Decrement $m \leftarrow m - 1 = 2 - 1 = \mathbf{1}$. Loop back!

- **Iteration 2 ($m = 1$):**

1. *Block "Extract c_m ":* The coefficient of the highest power z^{-1} in $B^{(1)}(z)$ is 6.60.

$$\mathbf{c_1 = 6.60}$$

2. *Block "Peel Basis Contribution":* Compute $c_1 B_1(z) = 6.60(-0.75 + 1.00z^{-1}) = -4.95 + 6.60z^{-1}$.

$$\begin{aligned} B^{(0)}(z) &= B^{(1)}(z) - c_1 B_1(z) \\ &= (1.20 - (-4.95)) + (6.60 - 6.60)z^{-1} = \mathbf{6.15} \end{aligned} \quad (84)$$

(Observe that z^{-1} is canceled identically, leaving a 0th-order scalar constant).

3. *Decision Diamond "m = 1?":* $m = 1 \implies \mathbf{YES}$.
4. *Block "Extract Final Constant c_0 ":* Since $B_0(z) \equiv 1$:

$$\mathbf{c_0 = B^{(0)}(z) = 6.15}$$

5. *TERMINAL SUCCESS Block:* Algorithm terminates successfully.

$$\{\mathbf{c_0 = 6.15, \quad c_1 = 6.60, \quad c_2 = 4.00}\}$$

Flowchart State Execution Summary Table

Table 3 summarizes the numerical state transitions across both iterations of the synthesis pipeline.

Table 3: Step-by-Step Flowchart State Transitions for Worked Example 20.2

Iteration	Stage (m)	Active Polynomial Input	Extracted Parameter	Stability / Check	Next State / Polynomial
Step-Down Algorithm (Figure 8)					
Iter 1	$m = 2$	$A_2(z) = 1 - 0.9z^{-1} + 0.2z^{-2}$	$k_2 = \alpha_2(2) = \mathbf{0.20}$	$ 0.20 < 1$ (Stable)	$A_1(z) = 1 - 0.75z^{-1}$ [$\alpha_1(1) = -0.75$]
Iter 2	$m = 1$	$A_1(z) = 1 - 0.75z^{-1}$	$k_1 = \alpha_1(1) = \mathbf{-0.75}$	$ -0.75 < 1$ (Stable)	$A_0(z) = 1, m = 1 \implies$ Terminate!
Numerator Peeling Algorithm (Figure 9)					
Iter 1	$m = 2$	$B^{(2)}(z) = 2 + 3z^{-1} + 4z^{-2}$	$c_2 = \text{Coeff}(z^{-2}) = \mathbf{4.00}$	$m \neq 1 \implies$ Continue	$B^{(1)}(z) = 1.2 + 6.6z^{-1}$
Iter 2	$m = 1$	$B^{(1)}(z) = 1.2 + 6.6z^{-1}$	$c_1 = \text{Coeff}(z^{-1}) = \mathbf{6.60}$	$m = 1 \implies$ Final scalar	$c_0 = B^{(0)}(z) = \mathbf{6.15}$

Verification of Exact Numerator Reconstruction:

$$\begin{aligned}
 \sum_{m=0}^2 c_m B_m(z) &= c_0 B_0(z) + c_1 B_1(z) + c_2 B_2(z) \\
 &= 6.15(1) + 6.60(-0.75 + z^{-1}) + 4.00(0.20 - 0.90z^{-1} + z^{-2}) \\
 &= (6.15 - 4.95 + 0.80) + (6.60 - 3.60)z^{-1} + 4.00z^{-2} \\
 &= \mathbf{2 + 3z^{-1} + 4z^{-2}} \equiv B(z) \quad (\mathbf{Q.E.D.}) \tag{85}
 \end{aligned}$$

Both flowcharts execute deterministically, yielding exact realization coefficients without root-finding approximations.